

E-Commerce

Java Selenium Automation for bstackdemo.com application

Selenium Automation Capstone Project Report

Name : **Sumana Chowdhury**

Batch : **SA2510014**

Role : **Automation Trainee**

Technology : **Selenium WebDriver with Java**

Framework : **TestNG + Maven**

Design Pattern : **Page Object Model(POM) with Page Factory**

Automation Techniques : **Screenshot Capture, Extent Reports**

GitHub Link : <https://github.com/Sumana-tech14/BrowserStackDemo>

Table of Contents :

1. Project Title
2. Domain
3. Application Under Test (AUT)
4. Project Objective
5. Tools and Technologies Used
6. Framework Architecture
7. Modules Covered
8. Automated Test Scenarios
9. Project Structure
10. Test Execution
11. Reporting
12. Key Features of the Framework
13. Challenges Faced
14. Test Script Screenshots
15. Console Screenshots
16. Screenshots During Test Execution
17. Extent Report Screenshots
18. Git/GitHub : Version Control
19. Conclusion

1. Project Title

Automation Testing Framework for **bstackdemo.com** using Java and Selenium WebDriver.

2. Domain

E-Commerce Web Application

3. Application Under Test (AUT)

Website : <https://bstackdemo.com/>

Description :

Bstackdemo.com is a sample e-commerce web application provided by **BrowserStack** designed for automation practice. It includes features such as :

- User authentication (Login)
- Product listing and filtering
- Add to cart functionality
- Cart Validation
- Checkout and order placement

4. Project Objective

To design and implement a scalable, maintainable, and modular automation testing framework using :

- Selenium WebDriver
- TestNG
- Apache Maven

The framework validates core functionalities of the **bstackdemo** e-commerce application.

5. Tools and Technologies Used

Tool/ Technology	Purpose
Java	Programming Language
Selenium WebDriver	Browser Automation
TestNG	Test Execution & Reporting
Maven	Dependency Management & Build Tool
Extent Reports	HTML Test Reporting
Git/GitHub	Version Control

6. Framework Architecture

Design Pattern Used :

Page Object Model (POM) with PageFactory

Framework Components :

- **BaseTest.java**
Initializes WebDriver and handles setup/teardown.
- **Page Classes**
 - LoginPage
 - ProductPage
 - CartPage
 - CheckoutPage
- **Utility Classes**
 - ScreenshotUtils
- **Test Classes**
 - LoginPageTest
 - ProductPageTest
 - CartPageTest
 - CheckoutPageTest
- **Testing.xml** Test Suite configuration and execution control.
- **ExtentReports** Integrated for detailed HTML reporting

7. Modules Covered

1. Login Module

- Validate login with valid credentials
- Validate login with empty fields

2. Product Module

- Validate Product listing
- Validate Product details
- Apply product filters

3. Cart Module

- Validate No item to cart
- Add single item to cart
- Add multiple items to cart
- Remove item from cart
- Validate cart count and total price

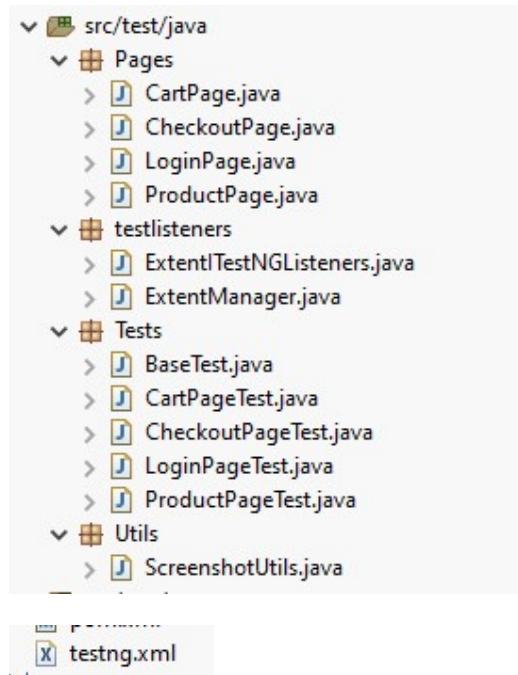
4. Checkout Module

- Place order with valid details
- Validate checkout without items
- Place order with incomplete details

8. Automated Test Scenarios

Test Case ID	Scenarios Description
TC_01	Login with empty username/ password
TC_02	Login with valid credentials
TC_03	Validate empty Cart
TC_04	Add single item to cart
TC_05	Add multiple items to cart
TC_06	Remove item from cart
TC_07	Place order with incomplete details
TC_08	Place order with valid details

9. Project Structure



10. Test Execution

- Using TestNG

Run **testing.xml** from IDE.

11. Reporting

- Extent Reports generates detailed HTML reports.
- TestNG default reports are generated
 - Reports include :
 - Test pass/Fail status
 - Execution time
 - Screenshots(on failure)
 - Logs

12. Key features of the framework

- Reusable Page Objects
- Centralized WebDriver management
- Configurable browser execution
- Screenshot capture on failure
- Clean and modular structure
- Easy scalability for future test cases

13. Challenges Faced

- Maintaining reusable page methods
- Managing test data efficiently
- Handling dynamic product loading

14. Test Script Screenshots

```
package Tests;

import java.time.Duration;

public class BaseTest {

    public static WebDriver driver;

    public LoginPage login;
    public ProductPage product;
    public CartPage cart;
    public CheckoutPage checkout;
    //public ScreenshotUtils screenshot;

    @BeforeMethod
    public void setup() {

        driver = new ChromeDriver();
        driver.manage().window().maximize();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));

        driver.get("https://bstackdemo.com/");

        // Initialize page objects
        login = new LoginPage(driver);
        product = new ProductPage(driver);
        cart = new CartPage(driver);
        checkout = new CheckoutPage(driver);
        //screenshot = new ScreenshotUtils();
    }

    @AfterMethod
    public void tearDown() {
        if (driver != null) {
            driver.quit();
        }
    }
}
```

```

package Pages;

import org.openqa.selenium.WebDriver;

public class LoginPage {

    WebDriver driver;

    // WebElements with locators for the login page
    @FindBy(xpath = "//span[@id='signin']")
    private WebElement SignIn;

    @FindBy(xpath = "//div[text()='Select Username']")
    private WebElement SelectUsername;

    @FindBy(xpath = "//div[text()='demouser']")
    private WebElement userName;

    @FindBy(xpath="//div[text()='Select Password']")
    private WebElement SelectPassword;

    @FindBy(xpath = "//div[text()='testingisfun99']")
    private WebElement password;

    @FindBy(id = "login-btn")
    private WebElement LoginBtn;

    // Constructor to initialize the WebElements
    public LoginPage(WebDriver driver) {
        this.driver = driver;
        PageFactory.initElements(driver, this);
    }

    // Method to get the current URL of the application
    public void getAppurl() {
        String currentUrl = driver.getCurrentUrl();
        Assert.assertEquals(currentUrl, "https://bstackdemo.com/", "Application URL is incorrect!");
        System.out.println("Application URL is correct!" + currentUrl);
    }
}

```

```

// Method to perform SignIn action
public void clickSignIn() {
    SignIn.click();
    //String expectedUrl = driver.getCurrentUrl();
    Assert.assertTrue(SelectUsername.isDisplayed(), "SignIn Button is not working");
    System.out.println("Successfully landed on the SignIn page");
}

//Method to select username
public void selectusername() {
    //SignIn.click();
    SelectUsername.click();
    userName.click();
}

// Method to select password
public void selectpassword() {
    SelectPassword.click();
    password.click();
}

// Method to perform login action
public void performlogin() {
    LoginBtn.click();
}
}

```

```

package Pages;

import java.util.List;

public class ProductPage {

    WebDriver driver;

    @FindBy(xpath = "//div[@class=\"shelf-container\"]")
    private List<WebElement> allProductList;

    @FindBy(xpath = "//p[@class=\"shelf-item__title\"]")
    private List<WebElement> allProductNames;

    // @FindBy(xpath = "//div[@class=\"shelf-item__buy-btn\"]")
    // private WebElement specificProduct;

    // Constructor to initialize the webElements
    public ProductPage(WebDriver driver) {
        this.driver = driver;
        PageFactory.initElements(driver, this);
    }

    // Method to get the total number of products displayed on the product page
    public void getTotalProducts() {
        int totalProducts = allProductList.size();
        System.out.println("Total number of products displayed on the product page: " + totalProducts);
    }

    // Method to get the details of each product displayed on the product page
    public void getAppProductDetail() {
        for(WebElement i : allProductList) {
            System.out.println(i.getText());
        }
        System.out.println("-----");
    }
}

```



```
package Pages;

import java.util.List;

public class CartPage {

    WebDriver driver;

    @FindBy(xpath = "//span[@class=\"bag bag--float-cart-closed\"]")
    private WebElement cartIcon;

    @FindBy(xpath = "//div[@class=\"buy-btn\"]")
    private WebElement continueShoppingBtn;

    @FindBy(xpath = "//div[text()=\"Checkout\"]")
    private WebElement checkoutBtn;

    // Locators for the products
    @FindBy(xpath = "//p[@class=\"shelf-item__title\"]")
    private List<WebElement> allProductNames;

    // Locator for the specific product to be added to the cart
    @FindBy(xpath = "(//div[@class=\"shelf-item__buy-btn\"])[3]")
    private WebElement iPhone12ProProduct;

    @FindBy(xpath = "(//div[@class=\"shelf-item__buy-btn\"])[12]")
    private WebElement GalaxyS20UltraProduct;

    @FindBy(xpath = "(//div[@class=\"shelf-item__buy-btn\"])[10]")
    private WebElement GalaxyS20Product;

    @FindBy(xpath = "(//div[@class=\"shelf-item__buy-btn\"])[17]")
    private WebElement Pixel4Product;

    // Locator for the list of all products
    @FindBy(xpath = "//div[@class=\"shelf-container\"]")
    private List<WebElement> allProductList;

    //Locator for number of products in the cart
    @FindBy(xpath = "//span[@class=\"bag__quantity\"]")
    private WebElement numberOfProductsInCart;

    // Locator for remove button in the cart
    @FindBy(xpath = "(//div[@class=\"shelf-item__del\"])[2]")
    private WebElement removeBtn;
```

```

// Constructor to initialize the WebElements
public CartPage(WebDriver driver) {
    this.driver = driver;
    PageFactory.initElements(driver,this);
}

// Method to click on the cart icon
public void clickCartIcon() {
    cartIcon.click();
}

// Method to click on the Continue Shopping button
public void clickContinueShopping() {
    continueShoppingBtn.click();
    System.out.println("Number of products in the list : " +numberOfProductsInCart.getText());
    String cartMsg = driver.findElement(By.xpath("//p[@class=\"shelf-empty\"]")).getText();
    System.out.println("Cart Message :"+cartMsg);
}

// Method to click on a specific product based on its name

public void checkSingleProduct(String productName) {
    for (WebElement i : allProductList) {
        if(i.getText().contains(productName)) {
            break;

        }
    }
    System.out.println("Product name is : "+productName);
    switch(productName) {
        case "iPhone 12 Pro":
            iPhone12ProProduct.click();
            break;
        case "Galaxy S20 Ultra":
            GalaxyS20UltraProduct.click();
            break;
        case "Galaxy S20":
            GalaxyS20Product.click();
            break;
        case "Pixel 4":
            Pixel4Product.click();
            break;
    }
}
}

```

```

        Thread.sleep(2000);
    } catch (InterruptedException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }
}

// Method to check the number of products in the cart
public void checkNumberOfProductsInCart() {
    System.out.println("Number of products in the list : " + numberOfProductsInCart.getText());

    if(numberOfProductsInCart.getText().equals("0")) {
        System.out.println("There is no products in the cart! Continue shopping to add products to the cart!");
    }
    else if(numberOfProductsInCart.getText().equals("1"))
    {
        System.out.println("There is only one product in the cart!");
    }
    else {
        System.out.println("There are multiple products in the cart!");
    }
}

// Method to remove the product from the cart and verify that the cart is empty
public void removeProductFromCart() {
    if(numberOfProductsInCart.getText()!="0") {
        String NumberOfProductsBeforeRemoving = numberOfProductsInCart.getText();
        System.out.println("Number of products in the cart before removing : " +NumberOfProductsBeforeRemoving);
        removeBtn.click();
        String NumberOfProductsAfterRemoving = numberOfProductsInCart.getText();
        System.out.println("Number of products in the cart after removing : " +NumberOfProductsAfterRemoving);
    }else {
        System.out.println("There is no products in the cart! Continue shopping to add products to the cart!");
    }
}

public void clickCheckout() {
    checkoutBtn.click();
    String checkoutMsg = driver.findElement(By.xpath("//legend[text()='Shipping Address']")).getText();
    Assert.assertEquals(checkoutMsg, "Shipping Address", "Checkout page was not successfully landed!");
    System.out.println("Successfully landed on the checkout page! Expected text is : " +checkoutMsg);
}
}

```

```

package Pages;

import org.openqa.selenium.By;

public class CheckoutPage {

    WebDriver driver;

    @FindBy(id = "firstNameInput")
    private WebElement firstName;

    @FindBy(id = "lastNameInput")
    private WebElement lastName;

    @FindBy(id = "addressLine1Input")
    private WebElement Address;

    @FindBy(id = "provinceInput")
    private WebElement State;

    @FindBy(id = "postCodeInput")
    private WebElement PostalCode;

    @FindBy(id = "checkout-shipping-continue")
    private WebElement SubmitBtn;

    // Constructor to initialize the WebElement
    public CheckoutPage(WebDriver driver) {
        // Initialize the WebDriver and WebElements using PageFactory
        this.driver = driver;
        PageFactory.initElements(driver, this);
    }

    // Method to fill in the shipping address details and submit the form
    public void fillShippingDetails(String fName,String LName,String address,String state,String postalcode) {
        firstName.sendKeys(fName);
        lastName.sendKeys(LName);
        Address.sendKeys(address);
        State.sendKeys(state);
        PostalCode.sendKeys(postalcode);
        SubmitBtn.click();
    }
}

```

```
    // Assertion to verify that the order was successfully placed
    public void verifyOrderConfirmation() {
        String expectedMsg = driver.findElement(By.xpath("//a[text()='\"StackDemo\"]")).getText();
        Assert.assertTrue(expectedMsg.contains("StackDemo"), "Form submission was not successful!");
        System.out.println(driver.findElement(By.id("confirmation-message")).getText());
    }
}
```

```
package Utils;

import java.io.File;

public class ScreenshotUtils {
    public static String getScreenshot(WebDriver driver, String name) {
        TakesScreenshot ts = (TakesScreenshot) driver;

        File src= ts.getScreenshotAs(OutputType.FILE);
        String folderpath = System.getProperty("user.dir") + "/BStackDemoScreenshots/";
        File folder= new File(folderpath);

        if(!folder.exists()) {
            folder.mkdirs();
        }

        String path = folderpath + name + System.currentTimeMillis() + ".png";
        File dest = new File(path);

        try {
            FileHandler.copy(src, dest);
        } catch (IOException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }

        return path;
    }
}
```

```

package testlisteners;

import org.testng.ITestContext;

public class ExtentTestListeners implements ITestListener {
    /*
     * ThreadLocal is a class in java.lang package provides a way to store values in per thread basis
     * It create separate instance of the value for each thread allowing each thread to have its own copy of the value
     * without interfering with other threads.
     *
     * It is useful for storing values that are specific to a single thread such as transaction or request specific data
     * ThreadLocal provides a simple and efficient way to store thread local variables without having to pass them as method arguments or store them as instance variables.
     */
    ExtentReports extent=ExtentManager.getInstance();
    ThreadLocal<ExtentTest> parentTest=new ThreadLocal<ExtentTest>();

    public void onStart(ITestResult result)
    {
        ExtentTest extentTest=extent.createTest(result.getMethod().getMethodName());
        parentTest.set(extentTest);
    }

    public void onSuccess(ITestResult result) {

        //parentTest.get().pass("Test Passed");
        parentTest.get().log(Status.PASS,result.getName()+"Got Successfully Pass!");
    }

    public void onFailure(ITestResult result)
    {
        String path =ScreenshotUtils.getScreenshot(BaseTest.driver, result.getMethod().getMethodName());
        parentTest.get().fail("Test Failed "+result.getThrowable().getMessage()).addScreenCaptureFromPath(path);
        System.out.println("Test Failure : "+result.getMethod().getMethodName());
        parentTest.get().log(Status.FAIL,result.getName()+"Test Fail");
    }

    public void onTestSkipped(ITestResult result) {
        parentTest.get().skip("Test Skipped "+result.getThrowable().getMessage());
    }

    public void onFinish(ITestContext context) {
        extent.flush();
    }
}

```

```

package testlisteners;

import java.text.SimpleDateFormat;

public class ExtentManager {

    public static ExtentReports extent;

    public static ExtentReports getInstance()
    {
        if(extent==null)
        {
            extent=createInstance();
            return extent;
        }
        else
        {
            return extent;
        }
    }

    static String rname;
    public static ExtentReports createInstance()
    {
        String timestamp=new SimpleDateFormat("yyyy.MM.dd.HH.mm.ss").format(new Date());
        rname="BrowserStackTestReport"+timestamp+".html";

        ExtentSparkReporter sparkReporter=new ExtentSparkReporter(System.getProperty("user.dir")+"/BStackDemoTestReports/"+rname);
        sparkReporter.config().setTheme(Theme.DARK);
        sparkReporter.config().setReportName("BrowserStackTestReport");
        sparkReporter.config().setDocumentTitle("BrowserStack_Report");

        ExtentReports extent=new ExtentReports();
        extent.attachReporter(sparkReporter);

        return extent;
    }
}

```



```

package Tests;

import org.openqa.selenium.By;

Run ALL
public class LoginPageTest extends BaseTest {

    @Test(priority=1)
    Run / Debug
    public void testInvalidUsernameAndPassword() throws InterruptedException {

        login.getAppUrl();
        login.clickSignIn();
        login.performLogin();
        Thread.sleep(2000);
        ScreenshotUtils.getScreenshot(driver, "EmptyLoginField");
        System.out.println(driver.findElement(By.xpath("//h3")).getText());
    }

    @Test(priority=2)
    Run / Debug
    public void testInvalidUserName() throws InterruptedException {

        login.getAppUrl();
        login.clickSignIn();
        login.selectpassword();
        login.performLogin();
        Thread.sleep(2000);
        ScreenshotUtils.getScreenshot(driver, "InvalidUserName");
        System.out.println(driver.findElement(By.xpath("//h3")).getText());
    }

    @Test(priority=3)
    Run / Debug
    public void testInvalidPassword() throws InterruptedException {

        login.getAppUrl();
        login.clickSignIn();
        login.selectusername();
        login.performLogin();
        Thread.sleep(2000);
        ScreenshotUtils.getScreenshot(driver, "InvalidPassword");
        System.out.println(driver.findElement(By.xpath("//h3[text()='Invalid Password']")).getText());
    }

    }

    @Test(priority=4)
    Run / Debug
    public void testValidLogin() {

        login.getAppUrl();
        login.clickSignIn();
        login.selectusername();
        login.selectpassword();
        login.performLogin();
        String expectedtext = driver.findElement(By.xpath("//span[text()='Logout']")).getText();
        Assert.assertEquals(expectedtext, "Logout", "Login was not successful!");
        System.out.println("Login was successful! Expected text is : " + expectedtext);
    }

    }
}

```

```

package Tests;

import org.testng.annotations.Test;

Run ALL
public class ProductPageTest extends BaseTest {
    @Test
    Run / Debug
    public void testProductPage() {

        login.clickSignIn();
        login.selectusername();
        login.selectpassword();
        login.performlogin();

        product.getTotalProducts();
        product.getAppProductDetail();
        // product.clickOnProduct();

    }
}

```

```

package Tests;

import org.testng.annotations.Test;

Run ALL
public class CartPageTest extends BaseTest {
    @Test
    Run / Debug
    public void testforEmptyCart() throws InterruptedException {

        login.clickSignIn();
        login.selectusername();
        login.selectpassword();
        login.performlogin();

        cart.clickCartIcon();
        cart.clickContinuesShopping();
        Thread.sleep(2000);
        ScreenshotUtils.getScreenshot(driver, "EmptyCart");

    }

    @Test
    Run / Debug
    public void testforSingleProduct() {

        login.clickSignIn();
        login.selectusername();
        login.selectpassword();
        login.performlogin();

        cart.checkSingleProduct("iPhone 12 Pro");
        cart.checkNumberOfProductsInCart();
        cart.clickCheckout();

    }

    @Test
    Run / Debug
    public void testforMultipleProducts() throws InterruptedException{

        login.clickSignIn();
        login.selectusername();
        login.selectpassword();
        login.performlogin();
    }
}

```

```

        cart.checkSingleProduct("iPhone 12 Pro");
        cart.checkSingleProduct("Galaxy S20");
        cart.checkSingleProduct("Pixel 4");
        cart.checkNumberOfProductsInCart();
        cart.clickCheckout();
    }

    @Test
    Run / Debug
    public void testforRemoveProduct() throws InterruptedException {

        login.clickSignIn();
        login.selectusername();
        login.selectpassword();
        login.performlogin();

        cart.checkSingleProduct("iPhone 12 Pro");
        cart.checkSingleProduct("Galaxy S20");
        cart.checkSingleProduct("Pixel 4");
        cart.removeProductFromCart();
        Thread.sleep(2000);
        ScreenshotUtils.getScreenshot(driver, "RemoveProduct");
        System.out.println("Product Successfully removed from cart");
        cart.clickCheckout();
    }
}

package Tests;

import org.testng.Assert;

Run ALL
public class CheckoutPageTest extends BaseTest {
    @Test(priority=2)
    Run / Debug
    public void testforValidInputs() {

        login.clickSignIn();
        login.selectusername();
        login.selectpassword();
        login.performlogin();

        cart.checkSingleProduct("iPhone 12 Pro");
        cart.checkSingleProduct("Pixel 4");
        cart.clickCheckout();

        checkout.fillShippingDetails("John", "Smith", "Main street", "NY", "40004");
        checkout.verifyOrderConfirmation();
    }

    @Test(priority=1)
    Run / Debug
    public void testMissingDetails() throws InterruptedException {

        login.clickSignIn();
        login.selectusername();
        login.selectpassword();
        login.performlogin();

        cart.checkSingleProduct("iPhone 12 Pro");
        cart.checkSingleProduct("Pixel 4");
        cart.clickCheckout();

        checkout.fillShippingDetails("John", "Smith", "", "NY", "40004");
        Thread.sleep(2000);
        ScreenshotUtils.getScreenshot(driver, "EmptyField");
        String expectedUrl = driver.getCurrentUrl();
        Assert.assertTrue(expectedUrl.contains("checkout"), "Address error message is not displayed!");
        System.out.println("Please fill out Address field");
    }
}

```



```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="Suite">

  <listeners>

    <listener class-name="testlisteners.ExtentITestNGListeners"> </listener>

  </listeners>

  <test thread-count="5" name="Test">
    <classes>
      <class name="Tests.LoginPageTest"/>
      <class name="Tests.ProductPageTest"/>
      <class name="Tests.CartPageTest"/>
      <class name="Tests.CheckoutPageTest"/>
    </classes>
  </test> <!-- Test -->
</suite> <!-- Suite -->
```

15. Console Screenshots

```
[RemoteTestNG] detected TestNG version 7.12.0
SLF4J(W): No SLF4J providers were found.
SLF4J(W): Defaulting to no-operation (NOP) logger implementation
SLF4J(W): See https://www.slf4j.org/codes.html#noProviders for further details.
Mar 24, 2026 12:48:11 PM org.openqa.selenium.manager.SeleniumManager lambda$runCommand$1
WARNING: Error sending stats to Plausible: error sending request for url (https://plausible.io/api/event)
Mar 24, 2026 12:48:15 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 146, returning the closest version; found: 145; Please update t
Application URL is correct!https://bstackdemo.com/
Successfully landed on the SignIn page
Invalid Username
Mar 24, 2026 12:48:42 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 146, returning the closest version; found: 145; Please update t
Application URL is correct!https://bstackdemo.com/
Successfully landed on the SignIn page
Invalid Username
Mar 24, 2026 12:49:03 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 146, returning the closest version; found: 145; Please update t
Application URL is correct!https://bstackdemo.com/
Successfully landed on the SignIn page
Invalid Password
Mar 24, 2026 12:49:21 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 146, returning the closest version; found: 145; Please update t
Application URL is correct!https://bstackdemo.com/
Successfully landed on the SignIn page
Login was successful! Expected text is : Logout
Mar 24, 2026 12:49:39 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 146, returning the closest version; found: 145; Please update t
Successfully landed on the SignIn page
Total number of products displayed on the product page: 1
Products
25 Product(s) found.
Order by
Select
Lowest to highest
Highest to lowest
iPhone 12
$799.00
or 9 x $ 88.78
Add to cart
iPhone 12 Mini
$699.00
or 9 x $ 77.67
Add to cart
```

iPhone 12 Pro Max
\$1099.00
or 9 x \$ 122.11
Add to cart
iPhone 12 Pro
\$999.00
or 5 x \$ 199.80
Add to cart
iPhone 11
\$599.00
or 8 x \$ 74.88
Add to cart
iPhone 11 Pro
\$699.00
or 3 x \$ 233.00
Add to cart
iPhone XS
\$549.00
or 4 x \$ 137.25
Add to cart
iPhone XR
\$499.00
or 5 x \$ 99.80
Add to cart
iPhone XS Max
\$649.00
or 7 x \$ 92.71
Add to cart
Galaxy S20
\$999.00
or 9 x \$ 111.00
Add to cart
Galaxy S20+
\$1199.00
or 9 x \$ 133.22
Add to cart
Galaxy S20 Ultra
\$1399.00
or 12 x \$ 116.58
Add to cart
Galaxy S10
\$899.00
or 5 x \$ 179.80
Add to cart

Galaxy S9
\$699.00
or 5 x \$ 139.80
Add to cart
Galaxy Note 20
\$999.00
or 7 x \$ 142.71
Add to cart
Galaxy Note 20 Ultra
\$1299.00
or 7 x \$ 185.57
Add to cart
Pixel 4
\$899.00
or 5 x \$ 179.80
Add to cart
Pixel 3
\$599.00
or 5 x \$ 119.80
Add to cart
Pixel 2
\$399.00
or 3 x \$ 133.00
Add to cart
One Plus 8
\$799.00
or 6 x \$ 133.17
Add to cart
One Plus 8T
\$899.00
or 6 x \$ 149.83
Add to cart
One Plus 8 Pro
\$899.00
or 6 x \$ 149.83
Add to cart
One Plus 7T
\$599.00
or 6 x \$ 99.83
Add to cart
One Plus 7
\$499.00
or 3 x \$ 166.33
Add to cart

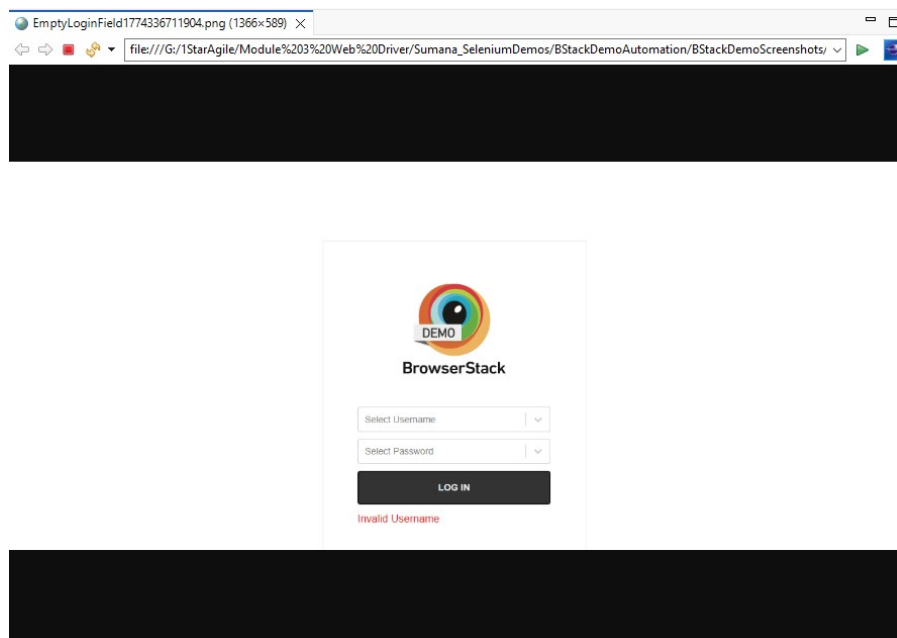
```

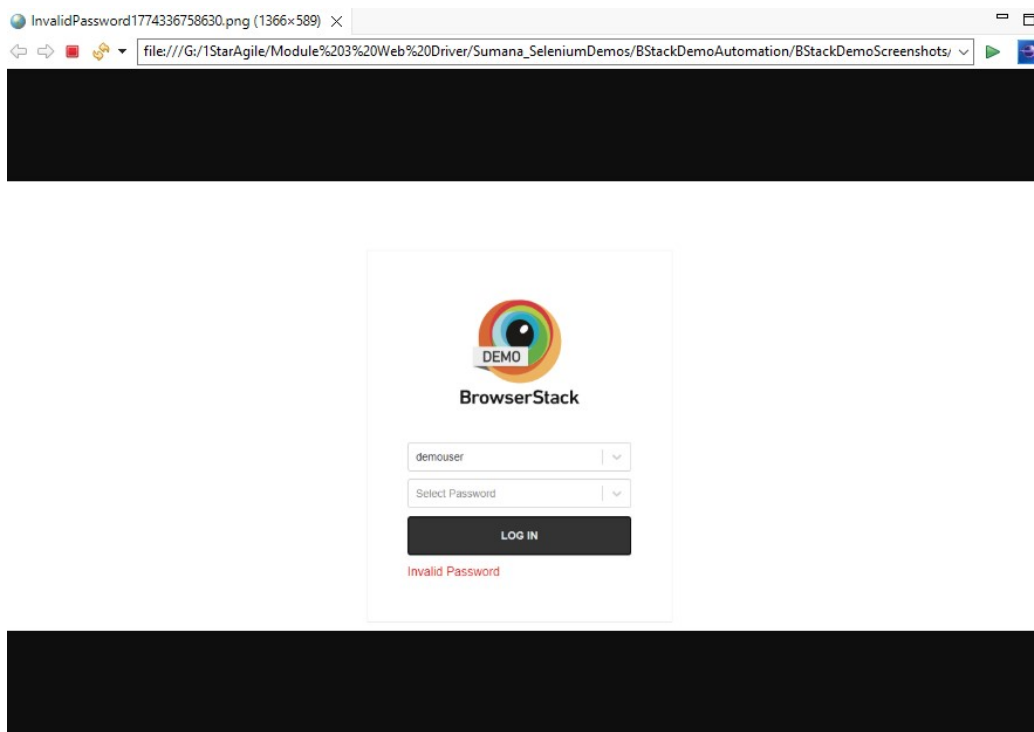
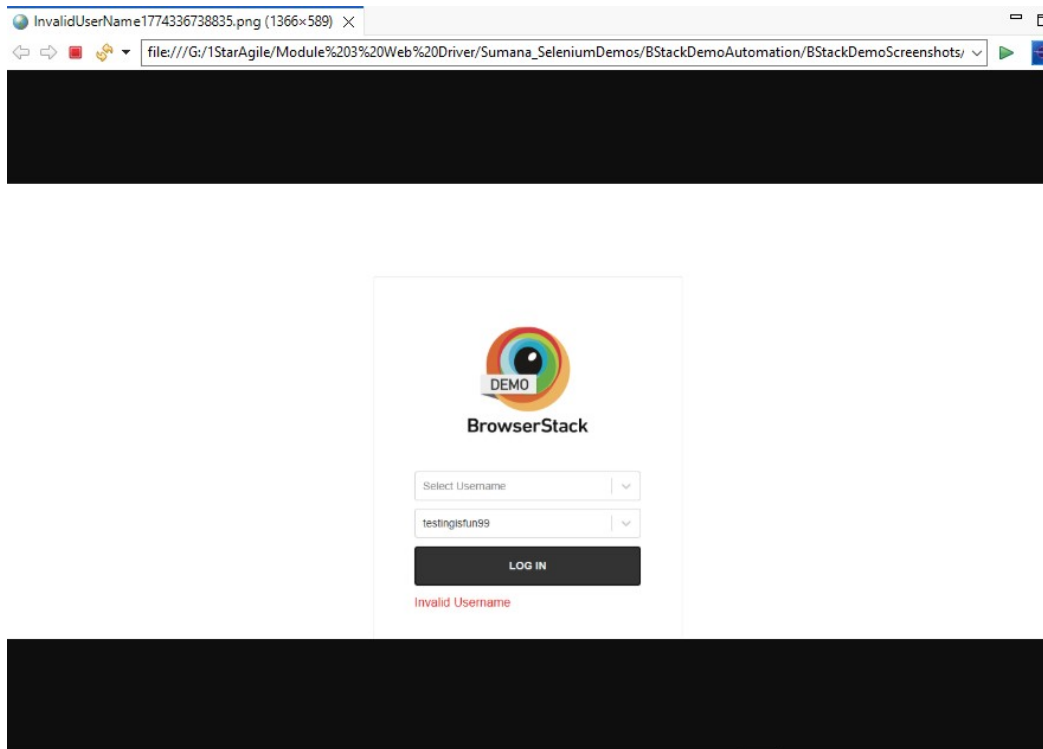
One Plus 6T
$429.00
or 3 x $ 143.00
Add to cart
-----
Mar 24, 2026 12:50:02 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 146, returning the closest version; found: 145; Please update 1
Successfully landed on the SignIn page
Number of products in the list : 0
Cart Message :Add some products in the bag
:)
Mar 24, 2026 12:50:19 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 146, returning the closest version; found: 145; Please update 1
Successfully landed on the SignIn page
Product name is : iPhone 12 Pro
Product name is : Galaxy S20
Product name is : Pixel 4
Number of products in the list : 3
There are multiple products in the cart!
Successfully landed on the checkout page! Expected text is : Shipping Address
Mar 24, 2026 12:50:39 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 146, returning the closest version; found: 145; Please update 1
Successfully landed on the SignIn page
Product name is : iPhone 12 Pro
Product name is : Galaxy S20
Product name is : Pixel 4
Number of products in the cart before removing : 3
Number of products in the cart after removing : 2
Product Successfully removed from cart
Successfully landed on the checkout page! Expected text is : Shipping Address
Mar 24, 2026 12:51:01 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 146, returning the closest version; found: 145; Please update 1
Successfully landed on the SignIn page
Product name is : iPhone 12 Pro
Number of products in the list : 1
There is only one product in the cart!
Successfully landed on the checkout page! Expected text is : Shipping Address
Mar 24, 2026 12:51:20 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 146, returning the closest version; found: 145; Please update 1
Successfully landed on the SignIn page
Product name is : iPhone 12 Pro
Successfully landed on the checkout page! Expected text is : Shipping Address
Please fill out Address field
Mar 24, 2026 12:51:43 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
Mar 24, 2026 12:51:43 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 146, returning the closest version; found: 145; Please update 1
Successfully landed on the SignIn page
Product name is : iPhone 12 Pro
Successfully landed on the checkout page! Expected text is : Shipping Address
Your Order has been successfully placed.

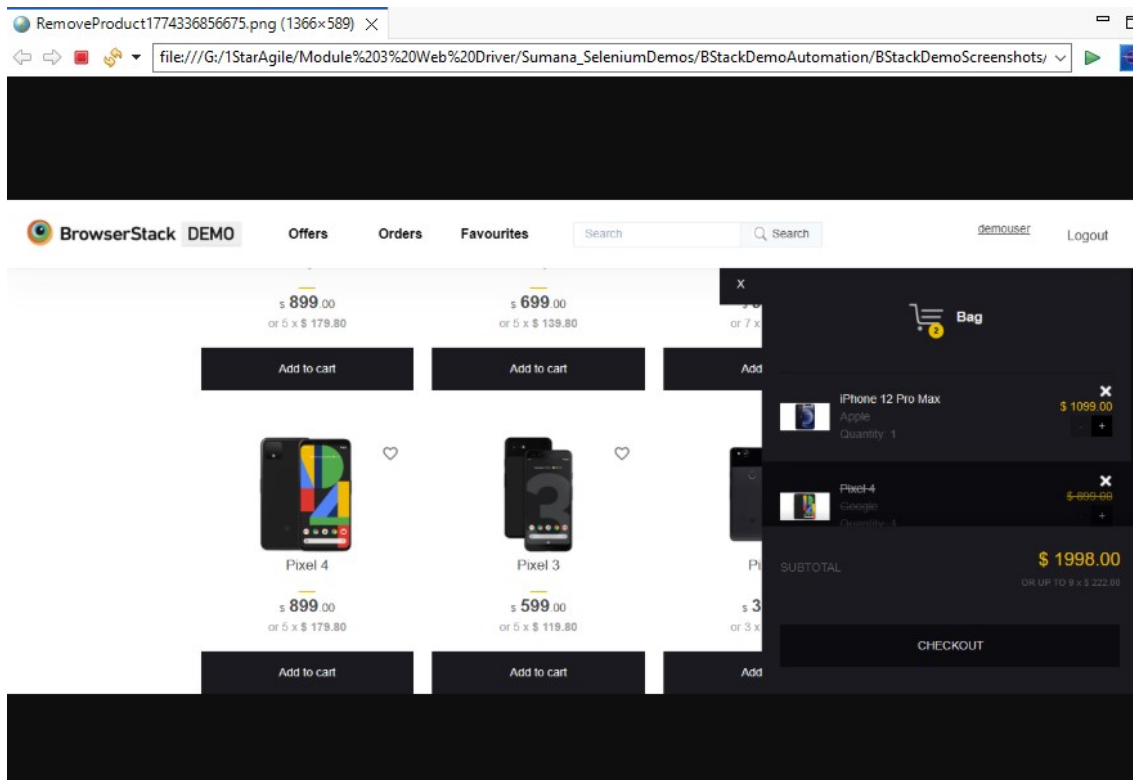
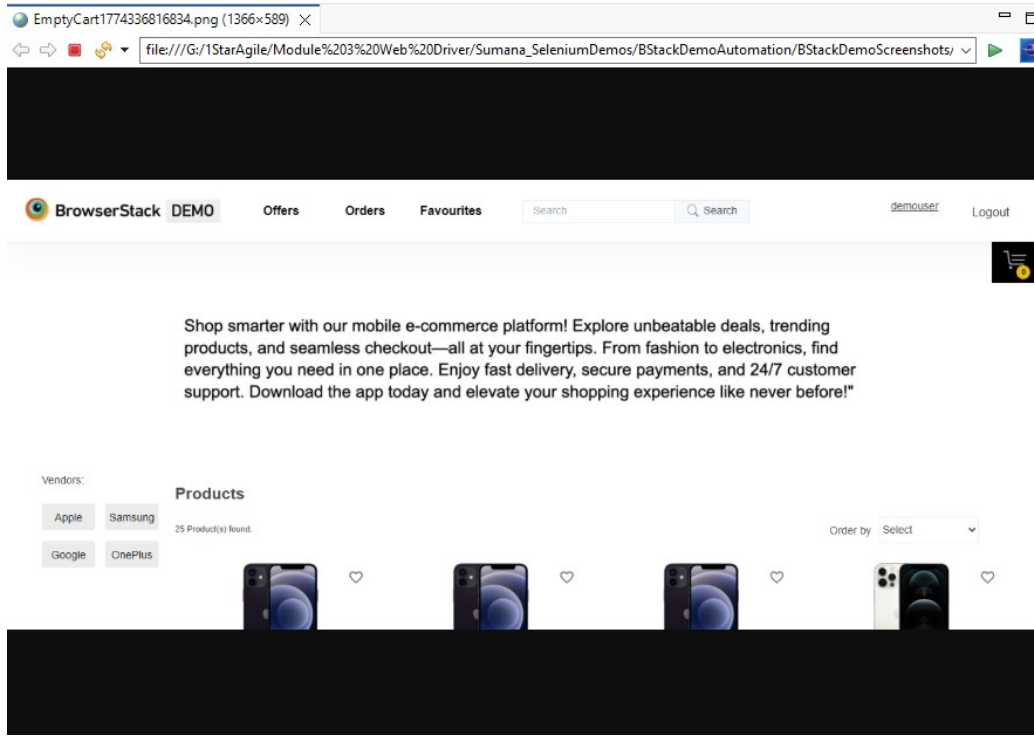
=====
Suite
Total tests run: 11, Passes: 11, Failures: 0, Skips: 0
=====

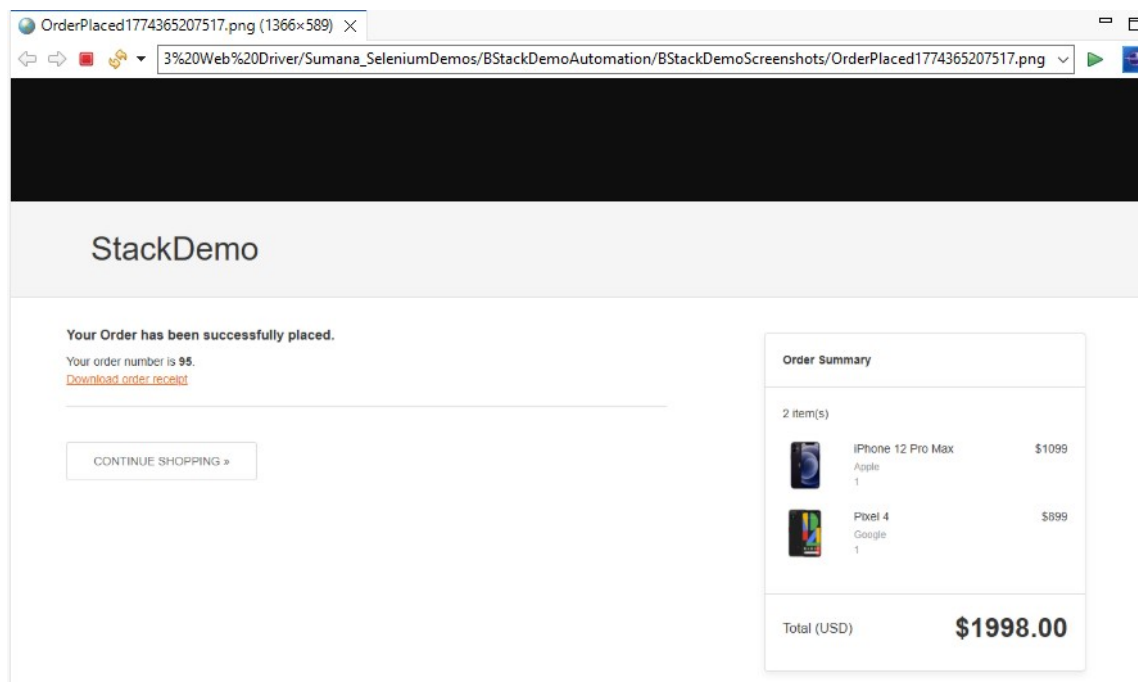
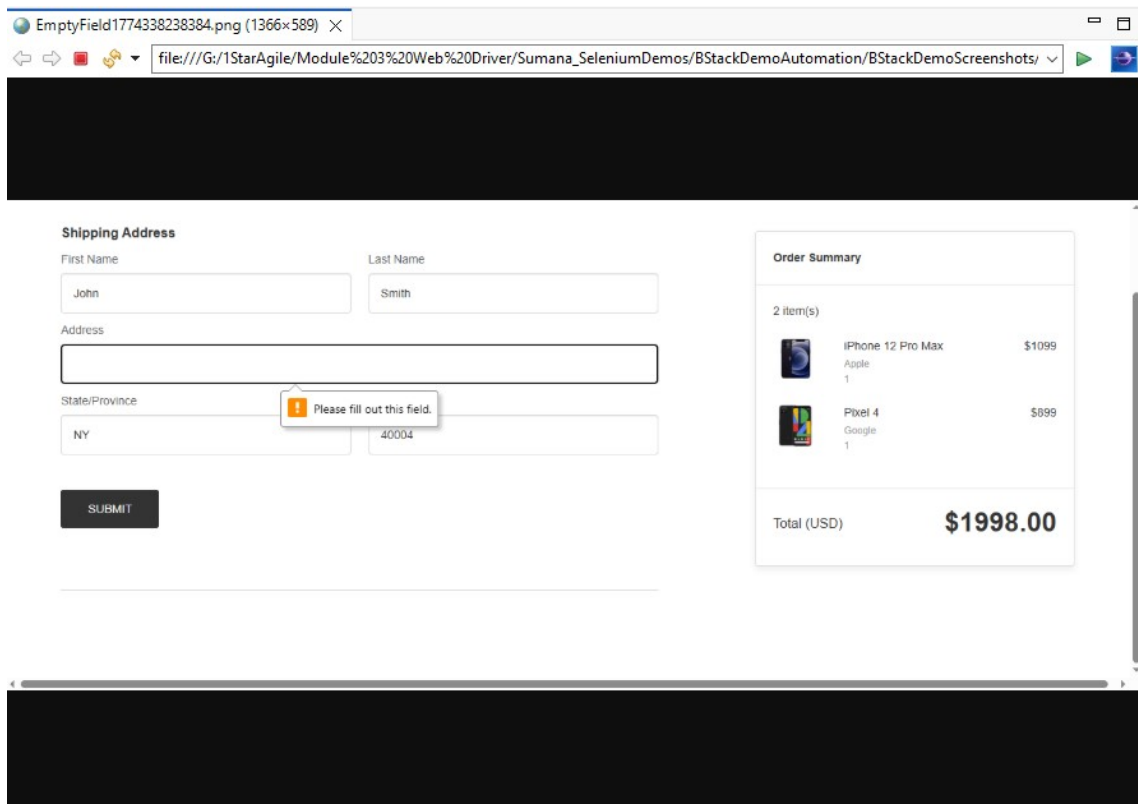
```

16. Screenshots during Test Execution









17. Extent Reports Screenshots

This screenshot displays the Extent Reports interface. On the left, a list of tests is shown, each with a status icon (green for passed, red for failed). The tests listed are:

- testInvalidUsernameAndPassword
- testInvalidUserName
- testInvalidPassword
- testValidLogin
- testProductPage
- testForEmptyCart
- testForMultipleProducts
- testForRemoveProduct
- testForSingleProduct
- testMissingDetails
- testForValidInputs

The right pane shows the details for the selected test, **testInvalidUsernameAndPassword**. It includes a status bar indicating the test passed, a timestamp of 12:48:31 pm, and a details section showing the test result: **testInvalidUsernameAndPasswordCor Successfully Pass!**

This screenshot displays the Extent Reports summary page. It includes a top navigation bar with the report title and date. The main content area is divided into several sections:

- Started:** Mar 24, 2026 12:48:04 pm
- Ended:** Mar 24, 2026 12:52:01 pm
- Tests Passed:** 11
- Tests Failed:** 0

Below these statistics, there are two large circular progress indicators, both showing 100% completion. The left indicator is labeled "Tests" and the right one is labeled "Log events".

At the bottom, there is a **Timeline** section showing a horizontal bar chart representing the duration of the tests. The timeline is divided into segments of different colors, indicating the duration of each test.

18. Git/GitHub : Version Control

Link : <https://github.com/Sumana-tech14/BrowserStackDemo>

The screenshot shows the GitHub interface for the repository 'BrowserStackDemo' by user 'Sumana-tech14'. The repository is public and has 1 branch and 0 tags. The commit history shows an initial commit by Sumana-tech14 33 minutes ago, with a commit hash of 008771a. The commit message is 'Initial commit'. The commit includes several files and folders: .settings, BStackDemoScreenshots, BStackDemoTestReports, src/test/java, target, test-output, .classpath, .project, and pom.xml. All files and folders were added in the initial commit.

File/Folder	Commit	Time
.settings	Initial commit	33 minutes ago
BStackDemoScreenshots	Initial commit	33 minutes ago
BStackDemoTestReports	Initial commit	33 minutes ago
src/test/java	Initial commit	33 minutes ago
target	Initial commit	33 minutes ago
test-output	Initial commit	33 minutes ago
.classpath	Initial commit	33 minutes ago
.project	Initial commit	33 minutes ago
pom.xml	Initial commit	33 minutes ago

19. Conclusion

The automation framework for bstackdemo.com was successfully designed and implemented using Java, Selenium WebDriver, TestNG, and Maven. The framework follows the Page Object Model design pattern, ensuring maintainability, scalability and reusability. It validates the core functionalities of the e-commerce application with structured test scenarios and detailed reporting.